

QUALIDADE E TESTES DE SOFTWARE

Aula prática: teste manual e teste automatizado de uma tela de login

Projeto base: HTML, CSS, JavaScript, Node.js, npm e Playwright

Objetivo geral da atividade

Compreender como uma funcionalidade simples, como uma tela de login, pode ser testada de forma manual e automatizada.
Aplicar conceitos de verificação, validação, caso de teste, resultado esperado, resultado obtido, defeito, falha e evidência de teste.
Executar testes automatizados utilizando Playwright e comandos npm.

Sumário

1. Apresentação da atividade prática
2. Conceitos essenciais de qualidade e testes
3. Projeto utilizado na aula
4. Regras de negócio da tela de login
5. Teste manual da funcionalidade de login
6. Registro de defeitos
7. Preparação do ambiente de automação
8. Passo a passo do npm install e Playwright
9. Testes automatizados com Playwright
10. Execução dos testes e interpretação dos resultados
11. Atividade prática para entrega
12. Modelo de relatório do aluno
13. Rubrica de avaliação
14. Referências

1. Apresentação da atividade prática

Nesta atividade, será utilizado um projeto simples de tela de login para demonstrar como uma equipe de qualidade pode testar uma funcionalidade de software. A tela possui dois campos, usuário e senha, e um botão para realizar o acesso.

A proposta é iniciar com testes manuais, registrando os cenários em uma tabela de casos de teste. Depois, os mesmos cenários serão transformados em testes automatizados utilizando Playwright.

Resultado esperado ao final da atividade

Entender o comportamento esperado da funcionalidade.
Criar e executar casos de teste manuais.
Registrar defeitos encontrados.
Instalar dependências com npm e executar testes automatizados.
Interpretar o relatório de testes automatizados.

2. Conceitos essenciais de qualidade e testes

2.1 Qualidade de software

Qualidade de software é a capacidade de um sistema atender às necessidades dos usuários e às regras definidas para o produto. Um software com qualidade não é apenas aquele que abre ou funciona. Ele precisa funcionar corretamente, ser fácil de usar, responder de forma adequada, proteger dados e apresentar mensagens claras.

2.2 Verificação e validação

Conceito	Pergunta principal	Exemplo no projeto de login
Verificação	Estamos construindo o sistema corretamente?	Analisar se o código valida usuário vazio e senha vazia.
Validação	Estamos construindo o sistema certo para o usuário?	Conferir se a tela permite acesso apenas com credenciais corretas.

2.3 Erro, defeito e falha

Termo	Significado simples	Exemplo	Quem percebe?
Erro	Ação humana incorreta.	O programador esqueceu de validar o campo senha.	Equipe técnica.
Defeito	Problema existente no código ou no produto.	O sistema não possui validação para senha vazia.	Equipe técnica ou equipe de testes.
Falha	Problema acontecendo durante o uso.	O usuário deixa a senha vazia e recebe mensagem incorreta.	Usuário ou testador.

2.4 Caso de teste

Caso de teste é um roteiro usado para verificar uma situação específica do sistema. Ele indica os dados usados, os passos executados, o resultado esperado, o resultado obtido e o status final.

Exemplo simples

Cenário: login com usuário e senha corretos.
Dados de entrada: usuário admin e senha 123456.
Resultado esperado: o sistema deve exibir a mensagem Login realizado com sucesso.

3. Projeto utilizado na aula

O projeto utilizado nesta atividade é composto por uma tela de login simples, desenvolvida com HTML, CSS e JavaScript. A automação dos testes será feita com Node.js, npm e Playwright.

```
projeto-login-teste/
|
```

```

|-- index.html
|-- style.css
|-- script.js
|-- package.json
|-- playwright.config.js
|
|-- tests/
|   |-- login.spec.js

```

3.1 Função de cada arquivo

Arquivo	Função no projeto
index.html	Estrutura visual da tela de login.
style.css	Aparência da página, cores, espaçamentos e layout.
script.js	Regras de validação do login.
package.json	Lista de dependências e scripts do projeto Node.js.
playwright.config.js	Configuração do Playwright para executar os testes.
tests/login.spec.js	Arquivo onde ficam os testes automatizados.

4. Regras de negócio da tela de login

Antes de testar, é necessário conhecer as regras esperadas para a funcionalidade. Sem regras definidas, não é possível saber se o comportamento do sistema está correto ou incorreto.

1. O campo usuário é obrigatório.
2. O campo senha é obrigatório.
3. O login só deve ser permitido com usuário admin e senha 123456.
4. Se o usuário estiver incorreto, o sistema deve bloquear o acesso.
5. Se a senha estiver incorreta, o sistema deve bloquear o acesso.
6. Se usuário e senha estiverem corretos, o sistema deve exibir mensagem de sucesso.
7. O campo senha deve ocultar os caracteres digitados.
8. As mensagens exibidas devem ser claras para o usuário.

5. Teste manual da funcionalidade de login

O teste manual será realizado primeiro para que o aluno compreenda a lógica do processo. Depois, esses mesmos cenários serão automatizados.

5.1 Tabela de casos de teste manuais

ID	Cenário	Usuário	Senha	Resultado esperado	Status
CT01	Login válido	admin	123456	Login realizado com sucesso.	
CT02	Usuário vazio		123456	O campo usuário é obrigatório.	
CT03	Senha vazia	admin		O campo senha é obrigatório.	
CT04	Usuário incorreto	aluno	123456	Usuário ou senha inválidos.	
CT05	Senha incorreta	admin	000000	Usuário ou senha inválidos.	
CT06	Usuário e senha vazios			O campo usuário é obrigatório.	
CT07	Usuário com espaços	admin	123456	Login realizado com sucesso.	
CT08	Usuário em maiúsculo	ADMIN	123456	Usuário ou senha inválidos.	
CT09	Campo senha oculto	admin	123456	A senha deve ficar oculta no campo.	

5.2 Como preencher resultado obtido e status

Depois de executar cada caso de teste, o aluno deve preencher o resultado obtido e comparar com o resultado esperado.

Situação	Como classificar	Exemplo
Resultado obtido igual ao esperado	Aprovado	Esperava sucesso e apareceu sucesso.
Resultado obtido diferente do esperado	Reprovado	Esperava senha obrigatória, mas apareceu usuário ou senha inválidos.
Não foi possível executar	Bloqueado	Projeto não abriu ou ambiente não funcionou.

6. Registro de defeitos

Quando um teste falha, o problema deve ser registrado de forma clara. Outra pessoa precisa conseguir reproduzir o defeito seguindo os passos informados.

6.1 Modelo de registro de defeito

Campo	Informação a preencher
ID do defeito	BUG01
Caso de teste relacionado	CT03
Funcionalidade	Login
Descrição do problema	Explicar o que aconteceu de errado.
Passos para reproduzir	Informar a sequência exata para encontrar o problema.
Resultado esperado	O que deveria acontecer.
Resultado obtido	O que aconteceu de verdade.
Severidade	Baixa, média, alta ou crítica.
Evidência	Print, relatório ou descrição do teste executado.

6.2 Severidade do defeito

Severidade	Significado	Exemplo
Baixa	Não impede o uso do sistema.	Erro de português em uma mensagem.
Média	Prejudica a experiência ou uma validação, mas não bloqueia totalmente.	Mensagem incorreta para senha vazia.
Alta	Impede uma funcionalidade importante.	Login não funciona mesmo com dados corretos.
Crítica	Compromete segurança ou funcionamento essencial.	Sistema permite acesso sem senha.

7. Preparação do ambiente de automação

Para executar os testes automatizados, o computador precisa ter Node.js instalado. O Node.js inclui o npm, que é o gerenciador de pacotes utilizado para baixar dependências do projeto.

Importante

npm install é o comando que baixa as dependências listadas no arquivo package.json.
 npx playwright install chromium baixa o navegador Chromium utilizado pelos testes.
 npx playwright test executa os testes automatizados.

7.1 Programas necessários

Programa	Uso	Observação
Node.js	Executar JavaScript fora do navegador e disponibilizar o npm.	Instalar versão LTS.
npm	Baixar dependências do projeto.	Já vem junto com o Node.js.
VS Code	Editar arquivos e abrir terminal.	Pode ser substituído por outro editor.
Playwright	Automatizar os testes no navegador.	Será instalado pelo npm.
Chromium	Navegador controlado pelo Playwright.	Será instalado pelo comando do Playwright.
Plugin vs code Live Server		

8. Passo a passo do npm install e Playwright

Esta seção apresenta o roteiro de instalação para executar o projeto entregue pelo professor.

8.1 Extrair o projeto

Extraia o arquivo ZIP do projeto em uma pasta simples, por exemplo:

```
C:\projetos\projeto-login-teste
```

Evite pastas com nomes muito longos, acentos ou caracteres especiais.

8.2 Abrir a pasta no VS Code

9. Abra o VS Code.
10. Clique em File > Open Folder.
11. Selecione a pasta projeto-login-teste.
12. Abra um novo terminal em Terminal > New Terminal.

8.3 Verificar se Node.js e npm estão funcionando

No terminal, execute:

```
node -v  
npm -v
```

Se aparecer uma versão para cada comando, o Node.js e o npm estão instalados corretamente.

8.4 Entrar na pasta do projeto pelo terminal

Se o terminal ainda não estiver dentro da pasta do projeto, use o comando cd:

```
cd C:\projetos\projeto-login-teste
```

8.5 Executar npm install

Execute o comando abaixo dentro da pasta do projeto:

```
npm install
```

Esse comando lê o arquivo package.json e instala as dependências necessárias dentro da pasta node_modules.

O que deve acontecer

O npm irá baixar as dependências do projeto.
Será criada ou atualizada a pasta node_modules.
Será criado ou atualizado o arquivo package-lock.json.
Se não aparecer mensagem de erro, a instalação foi concluída.

8.6 Instalar o navegador do Playwright

Após instalar as dependências, execute:

```
npx playwright install chromium
```

Esse comando instala o navegador Chromium que será controlado automaticamente pelo Playwright durante os testes.

8.7 Executar os testes automatizados

Para executar os testes sem mostrar o navegador, use:

```
npx playwright test
```

Para executar os testes mostrando o navegador na tela, use:

```
npx playwright test --headed
```

Para abrir o relatório visual dos testes, use:

```
npx playwright show-report
```

8.8 Se aparecer erro no PowerShell

Em alguns computadores, o PowerShell pode bloquear a execução do npm.ps1. Se aparecer erro informando que a execução de scripts está desabilitada, use uma das soluções abaixo.

Solução recomendada: usar o CMD

Abra o Prompt de Comando, também chamado de CMD, e execute os comandos normalmente:

```
cd C:\projetos\projeto-login-teste
npm install
npx playwright install chromium
npx playwright test --headed
```

Solução alternativa: usar npm.cmd e npx.cmd

No PowerShell, use os comandos com .cmd:

```
npm.cmd install
npx.cmd playwright install chromium
npx.cmd playwright test --headed
```

8.9 Quando usar npm init playwright@latest

O comando npm init playwright@latest é usado quando o projeto será criado do zero. No projeto entregue pelo professor, esse comando não é necessário, porque os arquivos já estão prontos.

Se for criar um projeto novo, as opções recomendadas são:

Pergunta exibida	Opção recomendada
TypeScript or JavaScript?	JavaScript
Where to put your end-to-end tests?	tests
Add a GitHub Actions workflow?	No
Install Playwright browsers?	Yes

9. Testes automatizados com Playwright

O Playwright permite controlar o navegador por meio de código. O teste automatizado abre a página, preenche os campos, clica no botão e verifica se a mensagem exibida é a esperada.

9.1 Exemplo de teste automatizado

```
// Importa os recursos de teste e validação do Playwright
const { test, expect } = require('@playwright/test');

// Endereço da página que será testada
const urlLogin = 'http://127.0.0.1:5500/index.html';

// Caso de teste automatizado para login válido
test('CT01 - Login válido', async ({ page }) => {

  // Abre a página de login no navegador controlado pelo Playwright
  await page.goto(urlLogin);

  // Localiza o campo usuário pelo ID e preenche com admin
  await page.locator('#usuario').fill('admin');

  // Localiza o campo senha pelo ID e preenche com 123456
  await page.locator('#senha').fill('123456');

  // Localiza o botão Entrar pelo ID e realiza o clique
  await page.locator('#btnEntrar').click();

  // Verifica se a mensagem exibida é exatamente a esperada
  await expect(page.locator('#mensagem')).toHaveText('Login realizado com sucesso.');
```

```
});
```

9.2 Explicação das principais linhas

Linha de código	O que ela faz
<code>const { test, expect } = require('@playwright/test');</code>	Importa as funções usadas para criar e validar testes.
<code>await page.goto(urlLogin);</code>	Abre a página que será testada.
<code>await page.locator('#usuario').fill('admin');</code>	Encontra o campo usuário e digita admin.
<code>await page.locator('#senha').fill('123456');</code>	Encontra o campo senha e digita 123456.
<code>await page.locator('#btnEntrar').click();</code>	Clica no botão Entrar.
<code>await expect(page.locator('#mensagem')).toHaveText(...);</code>	Confere se a mensagem exibida é a mensagem esperada.

9.3 Casos de teste automatizados sugeridos

ID	Cenário	Dados usados	Validação automática
CT01	Login válido	admin / 123456	Mensagem de sucesso.
CT02	Usuário vazio	vazio / 123456	Mensagem de usuário obrigatório.
CT03	Senha vazia	admin / vazio	Mensagem de senha obrigatória.
CT04	Usuário incorreto	aluno / 123456	Mensagem de usuário ou senha inválidos.
CT05	Senha incorreta	admin / 000000	Mensagem de usuário ou senha inválidos.
CT06	Usuário em maiúsculo	ADMIN / 123456	Mensagem de usuário ou senha inválidos.
CT07	Campo senha oculto	verificação do atributo type	Campo deve ter type password.

10. Execução dos testes e interpretação dos resultados

Ao executar os testes, o Playwright informa quais casos passaram e quais falharam. Um teste aprovado significa que o resultado obtido foi igual ao resultado esperado. Um teste reprovado indica diferença entre o comportamento esperado e o comportamento real.

10.1 Comandos principais

Comando	Finalidade
<code>npm install</code>	Instala as dependências listadas no package.json.
<code>npx playwright install chromium</code>	Instala o navegador Chromium usado nos testes.
<code>npx playwright test</code>	Executa os testes em modo padrão.
<code>npx playwright test --headed</code>	Executa os testes mostrando o navegador.
<code>npx playwright show-report</code>	Abre o relatório visual dos testes.

10.2 Como interpretar uma falha

Uma falha ocorre quando a validação automática não encontra o resultado esperado. Exemplo: o teste esperava a mensagem “O campo senha é obrigatório.”, mas o sistema exibiu “Usuário ou senha inválidos.”

Conclusão da falha

O teste não falhou porque o Playwright está errado.
 O teste falhou porque o comportamento do sistema ficou diferente da regra esperada.
 A equipe deve verificar se o erro está no sistema, na regra definida ou no próprio caso de teste.

11. Atividade prática para entrega

A atividade deverá ser realizada em grupo ou individualmente, conforme orientação do professor. O objetivo é testar a tela de login manualmente e automatizar parte dos testes com Playwright.

11.1 Etapas obrigatórias

13. Abrir o projeto de login no computador.
14. Ler as regras de negócio da funcionalidade.
15. Executar os casos de teste manuais.

16. Preencher resultado obtido e status de cada teste.
17. Registrar pelo menos um defeito, caso seja encontrado.
18. Executar npm install.
19. Executar npx playwright install chromium.
20. Executar npx playwright test --headed.
21. Abrir o relatório com npx playwright show-report.
22. Entregar o relatório final da atividade.

11.2 Entregáveis

- Tabela de casos de teste manuais preenchida.
- Registro de defeitos encontrados, se houver.
- Print ou evidência da execução dos testes automatizados.
- Print ou evidência do relatório do Playwright.
- Conclusão explicando o que foi aprendido com a atividade.

12. Modelo de relatório do aluno

O relatório final deve conter as informações abaixo.

Nome do aluno ou grupo:

Integrantes:

Data:

Sistema testado: Sistema de Login

Ferramenta de automação: Playwright

Navegador usado: Chromium

1. Objetivo da atividade
2. Regras de negócio analisadas
3. Casos de teste manuais
4. Resultado dos testes manuais
5. Defeitos encontrados
6. Passos realizados para instalar as dependências
7. Resultado dos testes automatizados
8. Evidências
9. Conclusão

13. Referências

PLAYWRIGHT. Installation. Documentação oficial. Disponível em: <https://playwright.dev/docs/intro>. Acesso em: 24 abr. 2026.

PLAYWRIGHT. Browsers. Documentação oficial. Disponível em: <https://playwright.dev/docs/browsers>. Acesso em: 24 abr. 2026.

NPM. npm-install. Documentação oficial. Disponível em: <https://docs.npmjs.com/cli/v8/commands/npm-install/>. Acesso em: 24 abr. 2026.

NPM. Specifying dependencies and devDependencies in a package.json file. Documentação oficial. Disponível em: <https://docs.npmjs.com/specifying-dependencies-and-devdependencies-in-a-package-json-file/>. Acesso em: 24 abr. 2026.